



Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University
(SPPU) ISO 21001:2018 Certified by TUV

Assignment No 3: Fractional Knapsack

Name : Yashodhan Prakash Suryawanshi

PRN : 125B2F004

Subject: Design and Analysis of Algorithms

Batch: TY- A4

Github Link : <https://github.com/Yasu225153/DAA-Lab-Submissions/tree/main/Assignment%20-%203>

1. Problem Statement:

A devastating flood has hit multiple villages in a remote area, and the government, along with NGOs, is organizing an emergency relief operation. A rescue team has a limited-capacity boat that can carry a maximum weight of W kilograms. The boat must transport critical supplies, including food, medicine, and drinking water, from a relief center to the affected villages.

Each type of relief item has:

- A weight (w_i) in kilograms.
- Utility value (v_i) indicating its importance (e.g., medicine has higher value than food).
- Some items can be divided into smaller portions (e.g., food and water), while others must be taken as a whole (e.g., medical kits).

2. Goals:

1. Implement the Fractional Knapsack algorithm to maximize the total utility value of the supplies transported.
2. Prioritize high-value items while considering weight constraints.
3. Allow partial selection of divisible items (e.g., carrying a fraction of food packets).
4. Ensure that the boat carries the most critical supplies given its weight limit W .

3. Course Objectives:

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

4. Course Outcomes:

After learning the course, students will be able to:

- Analyze the asymptotic performance of algorithms.
- Solve computational problems by applying suitable paradigms such as Greedy method.

5. Theory

The **Fractional Knapsack Problem** is a well-known optimization problem in which a set of items is given, with each item having a **weight (w_i)** and a **value (v_i)**. The objective is to select items within a knapsack of maximum capacity **W** such that the **total value is maximized**.

Unlike the **0/1 Knapsack Problem**, the Fractional Knapsack Problem allows an item to be divided into smaller portions. Therefore, if an entire item cannot fit into the remaining capacity, a suitable fraction of that item can be selected.

Greedy Algorithm

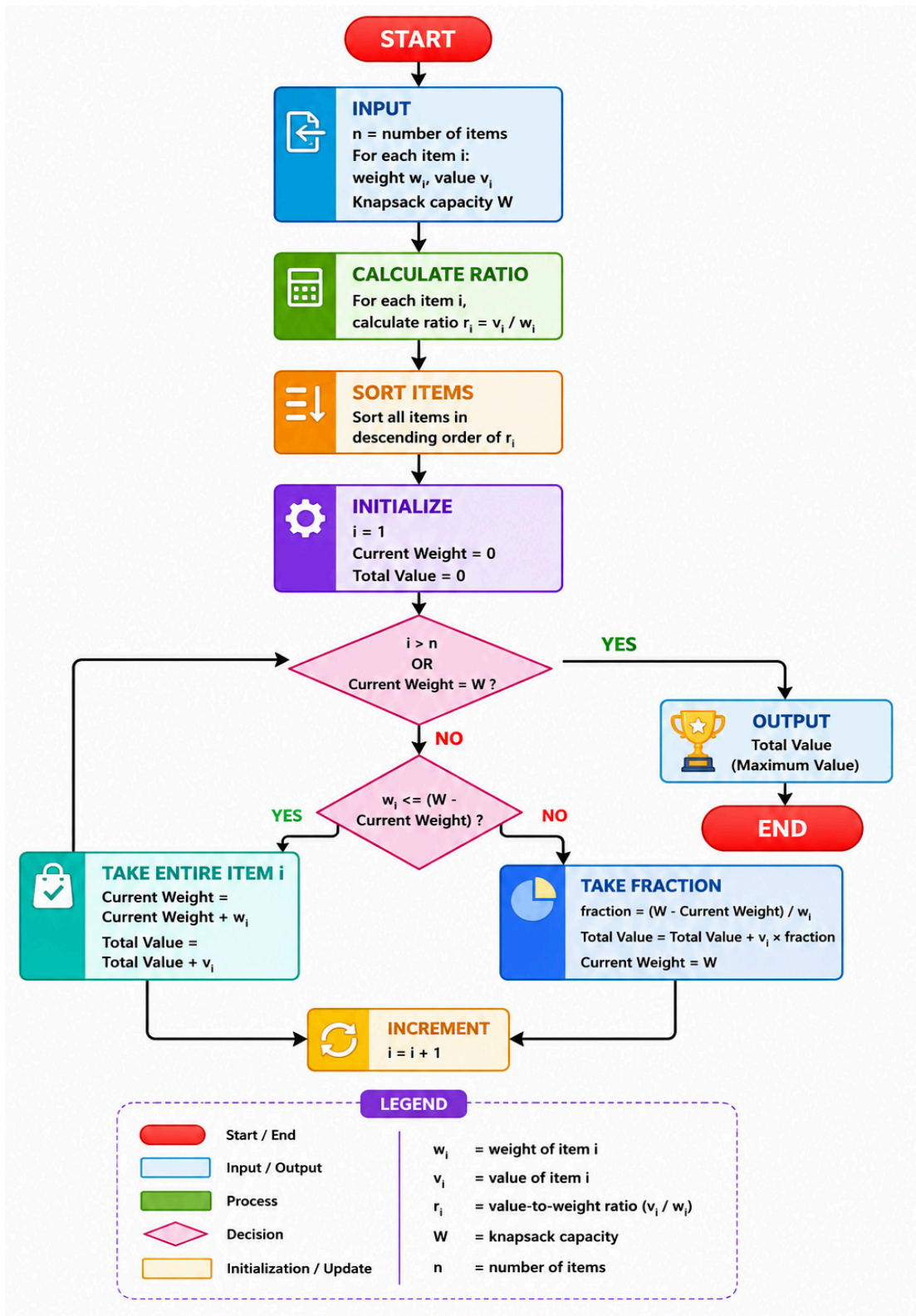
The Fractional Knapsack Problem can be solved optimally using the **Greedy approach**.

1. Calculate the **value-to-weight ratio (v_i / w_i)** for every item.
2. Sort all items in **descending order** of their value-to-weight ratio.
3. Start filling the knapsack with the item having the highest ratio.
4. Select the complete item if its weight fits within the remaining capacity.
5. If the complete item does not fit, select only the required fraction to fill the remaining capacity.
6. Continue until the knapsack reaches its maximum capacity.
7. For items that are **indivisible**, skip the item if it cannot be completely accommodated.

Working of the Greedy Approach

The greedy strategy works because of the following properties:

1. **Greedy Choice Property:** Selecting the item with the highest value-to-weight ratio at each step helps achieve the maximum possible total value.
2. **Optimal Substructure:** After selecting an item or a fraction of an item, the remaining capacity becomes a smaller Fractional Knapsack problem.
3. **Local to Global Optimization:** The best choice at each step contributes to the overall optimal solution.
4. **Fraction Selection:** Since items can be divided, unused capacity can be completely utilized by taking an appropriate fraction of an item.
5. **Efficiency:** Sorting the items based on their value-to-weight ratio makes the approach efficient for large numbers of items.



6. Time and Space Complexity

- Ratio computation: $O(n)$
- Sorting: $O(n \log n)$
- Selection: $O(n)$
- Total: $O(n \log n)$

7. Pseudo code

```
ALGORITHM MaxUtilFractional(items, W)
Input:
  items = list of (weight w, value v, isDivisible)
  W = maximum capacity
Output:
  Maximum total utility value
1. for each item in items do
2.   ratio ← value / weight
3. end for
4. Sort items in descending order of ratio
5. totalValue ← 0
6. remainingCapacity ← W
7. for each item in sorted items do
8.   if remainingCapacity = 0 then
9.     break
10.  end if
11. if item.weight ≤ remainingCapacity then
12.   totalValue ← totalValue + item.value
13.   remainingCapacity ← remainingCapacity - item.weight
14. else if item.isDivisible = true then
15.   fraction ← remainingCapacity / item.weight
16.   totalValue ← totalValue + (fraction × item.value)
17.   remainingCapacity ← 0
18. else
19.   // Item is indivisible and cannot fit
20.   continue
21. end if
22. end for
23. return totalValue
END ALGORITHM
```

6. Example:

Boat capacity: 25 kg

Item	Weight (kg)	Utility	Divisible	Taken	Value Obtained
Medical Kit	8	80	No	Full	80
Rice	12	36	Yes	Full	36
Blankets	15	15	Yes	Partial (if needed)	up to 15

Step 1: Calculate utility per kg

- Medical Kit: $80 / 8 = 10$
- Rice: $36 / 12 = 3$
- Blankets: $15 / 15 = 1$

Preference order: Medical Kit → Rice → Blankets

Step 2: Fill the boat

- Take Medical Kit fully: weight used = 8 kg, utility = 80, remaining capacity = $25 - 8 = 17$ kg

- Take Rice fully (weight 12 kg, divisible): fits completely, utility = 36, remaining capacity = $17 - 12 = 5$ kg
- Take Blankets partially (weight 15 kg, divisible): only 5 kg fits, utility = $15 \times (5/15) = 5$

Step 3: Total Utility

- Medical Kit: 80
- Rice: 36
- Blankets (partial): 5

Total Utility = 121

7. Output:

```
yashodhan@yashodhan-ASUS-TUF-Gaming-A17-FA706IHRB-FA706IHRB: ~/Desktop/125B2F004/Assignment - 3
~/Desktop/125B2F004/Assignment - 3
yashodhan@yashodhan-ASUS-TUF-Gaming-A17-FA706IHRB-FA706IHRB:~/Desktop/125B2F004/Assignment - 2$ cd ..
yashodhan@yashodhan-ASUS-TUF-Gaming-A17-FA706IHRB-FA706IHRB:~/Desktop/125B2F004$ cd "Assignment - 3"
yashodhan@yashodhan-ASUS-TUF-Gaming-A17-FA706IHRB-FA706IHRB:~/Desktop/125B2F004/Assignment - 3$ g++ fractionalKnapsack.cpp
yashodhan@yashodhan-ASUS-TUF-Gaming-A17-FA706IHRB-FA706IHRB:~/Desktop/125B2F004/Assignment - 3$ ./a.out
Enter number of items: 3

Item 1
Enter weight: 8
Enter value: 80
Is divisible? (1 = Yes, 0 = No): 0

Item 2
Enter weight: 12
Enter value: 36
Is divisible? (1 = Yes, 0 = No): 1

Item 3
Enter weight: 15
Enter value: 15
Is divisible? (1 = Yes, 0 = No): 1

Enter knapsack capacity: 25

Maximum Total Utility Value = 121.00
yashodhan@yashodhan-ASUS-TUF-Gaming-A17-FA706IHRB-FA706IHRB:~/Desktop/125B2F004/Assignment - 3$ g++
```

8. Conclusion:

The Fractional Knapsack Algorithm maximizes utility by prioritizing items with the highest value-to-weight ratio. In emergency relief logistics, this ensures that life-saving items like medicine kits are transported first, followed by food and water as space allows. The greedy algorithm is both efficient ($O(n \log n)$) and optimal for fractional cases, making it highly effective in disaster management where quick, resource-optimized decisions are critical.
